PROJECT REPORT

Title: Military Canteen Management System

Submitted By-
Name: Afiya Begum
Registration Number: 25BCE11165
Course: Fundamentals in AIML
Couse Code: CSA2001
Batch of: 2025

Table of Contents:

Introduction

The **Canteen Stores Department (CSD)** plays an indispensable role in providing essential, quality goods at subsidized rates to **Defence Personnel and their families**, ensuring a defined price advantage for this dedicated community. However, the sheer scale of CSD operations involving high volumes of diverse **inventory, numerous customer transactions, and complex supply chain logistics** often presents significant management challenges. Relying on traditional or semi-manual processes for record-keeping and stock management can lead to system bottlenecks. This project, the **Canteen Management System**, is developed to address these issues. It is designed as a **complete and versatile solution** aimed at automating and streamlining all core operational entries concerning **customers, products, sales, and purchase orders**, enhancing the overall efficiency and transparency of the canteen management.

Motivation Of The Project:

The motivation behind creating the project on **Canteen Management System** is to increase the efficiency in canteen management. In places where there is, for example, a canteen in the military, there is always a huge number of products, customers, and transactions that have to be handled effectively. When all these activities are carried out manually, there is always a high possibility of incurring certain issues that may affect the efficiency of canteen management. Some of these issues include data inconsistency, slower response in handling transactions, and difficulty in tracking customer and product information.

This project is also driven by the necessity of the increasing role played by digital technologies in modern-day management systems. The automation of canteen management enables administrators to easily add, edit, search, and delete entries pertaining to customer, product, and sales data. It also enables administrators in generating bills and maintaining inventory levels in real time. This makes it easier to avoid human errors, be transparent, and process transactions in an efficient manner.

The overall need to develop the Canteen Management System is to create a reliable, efficient, and user-friendly system that facilitates the effective management of the canteen, accuracy of data, and decision-making through the proper organization of records and reports.

Problem Statement

The current administrative and operational workflow of the Canteen Stores Department (CSD) is characterized by **fragmented data management**, primarily involving manual or disparate systems for inventory, sales, and customer tracking. This methodology presents significant systemic challenges:

1. **Inventory Inaccuracy:** Lack of a centralized database leads to real-time stock discrepancies, hindering effective purchasing decisions and increasing the risk of stockouts or wastage.
2. **Inefficient Transaction Processing:** Manual data entry for sales and invoice generation is slow and error-prone, reducing customer throughput.
3. **Untimely Reporting:** The inability to execute complex SQL queries and generate immediate, integrated reports prevents timely auditing, reconciliation, and strategic financial planning (e.g., tracking product due amounts).

Therefore, the objective is to design and implement a **Canteen Management System** utilizing Python and MySQL that establishes a **normalized, centralized relational database** to automate core CRUD operations, enforce data integrity, and provide instantaneous, accurate reporting necessary for efficient administrative and logistical control.

Objectives

1. To design a user friendly and systematic system to manage the canteen operations.
2. The operations of deletion and creation of records are streamlined to for the ease of the management.
3. Maintains an updated database of the customers (defence personnel and family).
4. To generate product and customer report for maintenance of records.

Existing Methods

Before the implementation of computerized systems, canteen management is mostly done through manual or semi-digital systems. These are conventional systems used in various small and medium-scale canteens. However, these conventional systems also generate certain problems in canteen management. There are various existing systems used in canteen management. One of these is the manual record-keeping system. In this system, canteen management is done through manual record-keeping. Here, manual registers are used to record customer details, product stock, and sales. Though this is an easy and cost-effective method, it is time-consuming and also includes certain errors. Maintaining large quantities of data in manual registers is also difficult.

Another conventional system used in canteen management is the spreadsheet-based system. In this system, Microsoft Excel is used for canteen management. In this system, data is organized in an efficient manner. However, it is also limited in certain areas, such as data management and errors in data duplication

Some canteens also use basic billing software. In this system, only bill generation is done. Here, customer management, product stock management, and sales are not considered.

Due to these limitations in existing systems, an efficient automated canteen management system is required.

Pros And Cons Of The Stated Method

1. Manual Record-Keeping System

Pros

- Easy and simple to implement without the need for any technical knowledge.
- Low cost since it only requires registers or notebooks.
- No need for computers and/or the internet.

Cons

- Too time-consuming when recording or searching data.
- More room for human error in calculation and record-keeping.
- Too hard to manage when dealing with huge data.
- Too slow and inefficient in making reports and/or maintaining inventories.

2.Spreadsheet-Based System

Pros

- Better organization of data compared to manual records.
- Allows basic calculations and sorting of information.
- Easy to edit and update records.
- Slightly faster than manual record keeping.

Cons

- Limited automation and integration between different records.
- Risk of data duplication or accidental deletion.
- Difficult to manage very large datasets.
- Requires basic computer knowledge to operate.
- Security and access control are limited.

3.Basic Billing Software / Semi-Automated Systems

Pros

- Time-saving in generating bills.
- Time-saving in processing transactions.
- Reduces calculation mistakes.
- Improves customer service through faster checkout.

Cons

- Limited to only billing operations.
- Inventory, customer, and reporting information may not be integrated.
- Limited flexibility in modifying the software.
- Inventory may still be updated manually.

System Requirements

Hardware:

- Processor: Intel Core i3 or above
- RAM: 4GB or above
- Hard Disk: 500GB or above
- Monitor: 15" LED

Software:

- Operating System: Windows/Linux/MacOS
- Programming Language: Python
- Database: MySQL

Methodology

The project is divided into two main parts:

1. **Frontend Development:** Using Python to create user interface (UI) for user interaction.
2. **Backend Development:**
   - Python programming to handle the logic of the system, such as purchasing, selling, and maintaining the records.
   - **Database:** MySQL for storing product details, customer information, and transaction history.

Functional Requirements

1. Customer Management Module (M1)

This module handles the maintenance of the customer database.

- Create Record: Allows the administrator to input new customer details, including ID, Name, Address, and Phone Number (create_record).
- Display Record: Retrieves and displays all customer records from the customer table (display_record).
- Search Record: Enables searching for a specific customer using their unique Customer ID (search_record).
- Modify Record (Update): Provides sub-functionality to update a customer's Name, Address, or Phone Number based on their ID (modify_record).
- Delete Record: Allows permanent removal of a customer record using their ID (delete_record).

2. Product/Inventory Management Module (M2)

This module manages the canteen's stock inventory and procurement data.

- Create Record: Allows input of new product details, including ID, Name, Company, Price, Quantity, Discount, Rate, and Purchase Date (create_rec). This function also updates the product_report table to track payment status.
- Display Record: Retrieves and displays the full list of products available in stock (display_rec).
- Search Record: Enables searching for a product using the Product ID (search_rec).
- Modify Record (Update): Provides sub-functionality to update specific product attributes like Name, Company, Price, Quantity, Discount, or Purchase Date (modify_rec).

- Delete Record: Allows permanent removal of a product record from the inventory (delete_rec).

3. Sales & Billing Management Module (M3)

This module manages sales transactions, inventory reduction, and bill generation.

- Add Customer Purchase (Sales): Records a new sale by linking the Customer ID, Product ID, and Quantity. Critically, it automatically calculates the final Amount and decrements the product's quantity in the product table (add_purchase).
- Modify Purchase Quantity: Allows modification of the quantity of an existing purchase, which then automatically adjusts the total sale amount and updates the product stock quantity (purch_qty).
- Delete Customer Purchase: Removes a recorded sale transaction (del_purchase).
- Generate Invoice: Creates a formal, printable invoice detailing all products, quantities, rates, and the total payable amount for a specific customer on a particular date

System workflow

The system is structured as a hierarchical, menu-driven command-line interface, ensuring a logical workflow for the administrator:

1. Main Menu: The user starts here, choosing between Products Report Generator or Administrator access (main_menu).
2. Administrator Menu: After selection, the user is presented with the three major operational categories: Customers Menu (M1), Products Menu (M2), or Canteen Sales Menu (M3) (admin).
3. Module Sub-Menus: Upon entering any of the three main module menus (M1, M2, or M3), the user is presented with a clear list of the available CRUD operations (Create, Display, Search, Modify, Delete) specific to that module.

4. Data Input/Output: Each operation prompts the user for specific input (e.g., ID, Name, Quantity) and provides clear formatted output (e.g., table display of records or confirmation messages).

Non-Functional Requirements

1.Usability

Simplicity and Intuitiveness: The entire system must be operable through the command-line interface (CLI) with clear, sequential, menu-driven navigation. All data entry prompts must be selfexplanatory to minimize administrator training time

2.Reliability (Error Handling)

Data Integrity & Robustness: The system must incorporate comprehensive error handling mechanisms (using Python's try-except blocks) to gracefully manage invalid inputs (e.g., non-numeric data for IDs or quantities) and connectivity issues with the MySQL database. Critical database operations (like sales and deletion) must be immediately confirmed or rejected with informative messages

3. Performance

Query Response Time: Database operations (Create, Read, Update, Delete) involving single-record lookups (e.g., search_rec, invoice generation) must execute and return results to the user within 2 seconds on the local host environment

4. Maintainability

Code Modularity and Clarity: The Python source code must be modular, utilizing distinct functions for each operation (e.g., create_rec, delete_record, prod_menu). This structure ensures that

future updates, debugging, or additions of new features (like reporting columns) can be isolated and implemented efficiently.

System Architecture

The Canteen Management System is built on a Three-Tier Architecture. This design was chosen to separate the user interface, the business logic, and the database, making the system easier to maintain and upgrade**.**

Architecture Components

1. Presentation Tier (User Interface):
   - Component: Command Line Interface (CLI).
   - Function: This is the visual layer where the administrator interacts with the system. It handles the menu displays (main_menu, cust_menu) and captures user inputs. It sends this data to the Logic Tier and displays the results (tables, success messages) back to the user.
2. **Logic Tier (Application Logic):** ◦

   Component: Python Functions.
   - Function: This is the "brain" of the application. It contains all the functions (e.g., add_purchase, create_rec) that process data. It performs calculations (like total bill amounts), validates user inputs (checking for integers vs. strings), and constructs the SQL commands**.**
3. **Data Tier (Storage):**
   - Component: MySQL Database.
   - Function: This layer stores all the persistent data in relational tables (customer, product, purchase). It executes the SQL

queries sent by the Python logic to fetch, update, or delete records.

Design Diagrams

This section visually represents the structure and behavior of the system.

Workflow Diagram

The following diagram illustrates the logical flow of the user interaction, moving from the Main Menu to specific administrative sub-menus.

Flow Description:

- **Start: The user begins at the Main Menu.**
- **Branch: They can choose "Products Report" (Read-only) or "Administrator" (Full Access).**
- **Action: Inside the Admin section, the flow splits into three functional paths: Customer Management, Product Inventory, and Canteen Sales.**
- **End: Each path leads to specific CRUD operations (Create, Read, Update, Delete) before returning to the menu.**

Entity-Relationship (ER) Diagram

This diagram defines the database schema and the relationships between the entities.

Entities & Relationships:

- **Customer:** Stores personal details (cust_id is the Primary Key).
- **Product**: Stores inventory details (p_id is the Primary Key).

- **Purchase**: The transaction table. It has a Many-to-One relationship with both Customer and Product (One customer can make many purchases; One product can be sold many times).
- **Product_Report**: Linked to Product, tracking financial status.

Sequence Diagram (Add Purchase)

This diagram details the sequence of events when a new sale is recorded, highlighting the system's internal logic.

Sequence of Events:

1. **User: Enters Customer ID and Product ID.**
2. **System: Validates that the input is a valid integer.**
3. **System: Queries the database to Insert the sale record.**
4. **Database: Updates the purchase table.**
5. **System: Immediately queries the database to Update the stock.**
6. **Database: Decrements p_qty in the product table.**
7. **System: Displays "Record Added" to the user.**

Design Decisions & Rationale

During the development of this project, several key technical decisions were made to ensure the system met the requirements for efficiency and reliability.

Choice of Technology

- Why Python? Python was selected for the logic layer because of its simplicity and the powerful mysql.connector library. This

allowed for rapid development of the CRUD functions without complex overhead.

- Why MySQL? A Relational Database Management System (RDBMS) was essential because the data (Customers, Products, Sales) is inherently structured and related. MySQL provides the necessary data integrity constraints (Primary and Foreign Keys) that a simple file-based system (like CSV) could not offer.

Architectural Decisions

- Modular Function Design: Instead of writing one giant script, the code is broken down into specific functions for each task (e.g., prod_menu, search_record). This makes the code readable and allows for easier debugging. If the "Search" feature breaks, I only need to fix the search_record function, not the entire program.
- Menu-Driven Interface: A text-based menu system was chosen over a Graphical User Interface (GUI) to prioritize performance and compatibility. It allows the system to run on any computer with Python installed, consuming minimal resources while remaining fast and responsive.

Data Handling Strategy

- Atomic Transactions: For the sales module, I decided to execute the "Record Sale" and "Update Inventory" queries in immediate succession followed by a single commit(). This decision ensures that the inventory count always matches the actual sales records, preventing stock discrepancies.

Code

```
#CANTEEN MANAGEMENT SYSTEM----------------------------------------
----------------------------------------------------------------------------------------
----------------------------------


try:
    import mysql.connector
    mydb=mysql.connector.connect(host='localhost', user='root',
password='tr!angulum3', database='canteen')
    mycursor=mydb.cursor()
except MySQLError:
    print("\nPlease check MYSQL connectivity !")
'''try:
    mycursor.execute("use PROJECT2025")
except:
    pass
'''


#INSTALLMENT------------------------------------------------------------------------
----------------------------------------------------------------------------------------
---------------------------------------


def tables():
    mycursor.execute('create table product(p_id int primary key, p_name
varchar(45), p_company varchar(45), p_price float, p_qty int, p_discount float,
p_rate float, p_purchasedate date')
```

```
    myscursor.execute('create table customer(cust_id int primary key, cust_name varchar(40), address varchar(50), phone char(10))')
    mycursor.execute('create table purchase(cust_id int, cust_name varchar(40), p_id int, p_name varchar(45), p_company varchar(30), rate float, quantity int, amount float, sale_date date)')

#PRODUCTS MENU STARTS------------------------------------------------------------
------------------------------------------------------------------------------------
------------------------------------------------------------

def create_rec():
    while True:
        try:
            prod_id=int(input("PRODUCT ID: "))
            prod_name=input("NAME OF PRODUCT: ")
            prod_company=input("NAME OF PRODUCT COMPANY: ")
            prod_price=float(input("PER UNIT PRODUCT PRICE: "))
            prod_quan=int(input("PRODUCT QUANTITY: "))
            prod_discount=float(input("PRODUCT DISCOUNT: "))
            prod_rate=float(input("PRODUCT SELLING RATE: "))
            prod_date=input("PRODUCT PURCHASE DATE (yyyy-mm-dd): ")
            prod_payment=input("Is the outstanding payment made? (Yes/No): ")
            if prod_payment.upper()=="NO":
                amount_left=int(input("Enter the due amount for product purchase: "))
            else:
                amount_left=0
        except ValueError:
```

```
            print("Incorrect value entered!")
        else:
            data1=(prod_id, prod_date, prod_payment, amount_left)
            data=(prod_id, prod_name, prod_company, prod_price, prod_quan,
prod_discount, prod_date)
            sql1='insert into product_report values(%s,%s,%s,%s)'
            sql='insert into product values(%s,%s,%s,%s,%s,%s,%s)'
            mycursor.execute(sql,data)
            mycursor.execute(sql1,data1)
            mydb.commit()
            opt=input("Do you want to enter more data?(y/n): ")
            if opt=="n":
                exit
            print("\nRecord added !!")


def display_rec():
    d={}
    j=1
    try:
        mycursor.execute("select * from product")
        dt=["PRODUCT ID", "PRODUCT NAME", "PRODUCT COMPANY",
"PRODUCT PRICE(PER UNIT)", "QUANTITY", "DISCOUNT", "RATE",
"PURCHASE DATE"]
        for i in mycursor:
            data = [i[0], i[1], i[2], i[3],i[4],i[5],i[6],i[7]]
            d[j]=data
            j+=1
```

```
        print ("{:<15} {:<15} {:<18} {:<22} {:<15} {:<15} {:<15}
{:<15}".format("PRODUCT ID", "PRODUCT NAME", "PRODUCT
COMPANY", "PRODUCT PRICE(PER UNIT)", "QUANTITY",
"DISCOUNT", "RATE", "PURCHASE DATE"))
        for k, v in d.items():
            a, b, c, h, e, f, g=v
            print("{:<15} {:<15} {:<18} {:<24} {:<15} {:<14} {:<14}
{:<15}".format(a, b, c, h, e, f, str(g)))
    except:
        print("Error encountered in displaying record from table!")


def search_rec():
    d={}
    j=1
    try:
        id=int(input("Enter the product id:"))
        mycursor.execute("select * from product where p_id="+str(id))
        dt=["PRODUCT ID", "PRODUCT NAME", "PRODUCT COMPANY",
"PRODUCT PRICE(PER UNIT)", "QUANTITY", "DISCOUNT",
"PURCHASE DATE"]
        for i in mycursor:
            data = [i[0], i[1], i[2], i[3],i[4],i[5],i[6],i[7]]
            d[j]=data
            j+=1
        print ("{:<15} {:<15} {:<18} {:<22} {:<15} {:<15} {:<15}
{:<15}".format("PRODUCT ID", "PRODUCT NAME", "PRODUCT
COMPANY", "PRODUCT PRICE(PER UNIT)", "QUANTITY",
"DISCOUNT", "RATE", "PURCHASE DATE"))
        for k, v in d.items():
```

```
            a, b, c, h, e, f, g=v
            print("{:<15} {:<15} {:<18} {:<24} {:<18} {:<15} {:<15}
{:<15}".format(a, b, c, h, e, f, str(g)))
    except:
        print("Error encountered in displaying record from table!")


def modify_rec():
    ch=0
    while ch!=7:
        print("="*30+"MODIFY PRODUCT-RECORD MENU"+"="*30)
        print("1. MODIFY PRODUCT NAME")
        print("2. MODIFY PRODUCT COMPANY")
        print("3. MODIFY PRODUCT PRICE")
        print("4. MODIFY PRODUCT QUANTITY")
        print("5. MODIFY PRODUCT DISCOUNT")
        print("6. MODIFY PRODUCT PURCHASE DATE")
        print("7. BACK TO PRODUCT MENU")
        print("="*86)
        ch=int(input("Enter your choice (1-7):"))
        if ch==1:
            prod_name()
        elif ch==2:
            prod_comp()
        elif ch==3:
            prod_price()
        elif ch==4:
```

```
            prod_quan()
        elif ch==5:
            prod_disc()
        elif ch==6:
            prod_purchase()
        elif ch==7:
            exit
        else:
            print("Incorrect choice! Please enter again!")


def prod_name():
    try:
        id=int(input("Enter the product id:"))
        pname=input("Enter the new product name:")
    except  ValueError:
        print("Incorrect value entered!")
    else:
        query="Update product set p_name=%s where p_id=%s"
        mycursor.execute(query,(pname, id))
        mydb.commit()
        print("\nThe modified data is:")
        d={}
        j=1
        mycursor.execute("select * from product where p_id="+str(id))
        dt=["PRODUCT ID", "PRODUCT NAME", "PRODUCT COMPANY",
"PRODUCT PRICE(PER UNIT)", "QUANTITY", "DISCOUNT",
"PURCHASE DATE"]
```

```
        for i in mycursor:
            data = [i[0], i[1], i[2], i[3],i[4],i[5],i[6],i[7]]
            d[j]=data
            j+=1
        print ("{:<15} {:<15} {:<18} {:<22} {:<15} {:<15} {:<15}
{:<15}".format("PRODUCT ID", "PRODUCT NAME", "PRODUCT
COMPANY", "PRODUCT PRICE(PER UNIT)", "QUANTITY",
"DISCOUNT", "RATE", "PURCHASE DATE"))
        for k, v in d.items():
            a, b, c, h, e, f, g=v
            print("{:<15} {:<15} {:<18} {:<24} {:<18} {:<15} {:<15}
{:<15}".format(a, b, c, h, e, f, str(g)))



def prod_comp():
    try:
        id=int(input("Enter the product id:"))
        pcomp=input("Enter the new product company:")
    except ValueError:
        print("Incorrect value entered!")
    else:
        query="Update product set p_company=%s where p_id=%s"
        mycursor.execute(query, (pcomp, id))
        mydb.commit()
        print("\nThe modified data is:")
        d={}
        j=1
```

```
        mycursor.execute("select * from product where p_id="+str(id))
        dt=["PRODUCT ID", "PRODUCT NAME", "PRODUCT COMPANY",
"PRODUCT PRICE(PER UNIT)", "QUANTITY", "DISCOUNT",
"PURCHASE DATE"]
        for i in mycursor:
            data = [i[0], i[1], i[2], i[3],i[4],i[5],i[6],i[7]]
            d[j]=data
            j+=1
        print ("{:<15} {:<15} {:<18} {:<22} {:<15} {:<15} {:<15}
{:<15}".format("PRODUCT ID", "PRODUCT NAME", "PRODUCT
COMPANY", "PRODUCT PRICE(PER UNIT)", "QUANTITY",
"DISCOUNT", "RATE", "PURCHASE DATE"))
        for k, v in d.items():
            a, b, c, h, e, f, g=v
            print("{:<15} {:<15} {:<18} {:<24} {:<18} {:<15} {:<15}
{:<15}".format(a, b, c, h, e, f, str(g)))



def prod_price():
    try:
        id=int(input("Enter the product id:"))
        pprice=input("Enter the new product quantity:")
    except ValueError:
        print("Incorrect value entered!")
    else:
        query="Update product set p_price=%s where p_id=%s"
        mycursor.execute(query, (pprice, id))
        mydb.commit()
```

```
        print("\nThe modified data is:")
        d={}
        j=1
        mycursor.execute("select * from product where p_id="+str(id))
        dt=["PRODUCT ID", "PRODUCT NAME", "PRODUCT COMPANY",
"PRODUCT PRICE(PER UNIT)", "QUANTITY", "DISCOUNT",
"PURCHASE DATE"]
        for i in mycursor:
            data = [i[0], i[1], i[2], i[3],i[4],i[5],i[6],i[7]]
            d[j]=data
            j+=1
        print ("{:<15} {:<15} {:<18} {:<22} {:<15} {:<15} {:<15}
{:<15}".format("PRODUCT ID", "PRODUCT NAME", "PRODUCT
COMPANY", "PRODUCT PRICE(PER UNIT)", "QUANTITY",
"DISCOUNT", "RATE", "PURCHASE DATE"))
        for k, v in d.items():
            a, b, c, h, e, f, g=v
            print("{:<15} {:<15} {:<18} {:<24} {:<18} {:<15} {:<15}
{:<15}".format(a, b, c, h, e, f, str(g)))



def prod_quan():
    try:
        id=int(input("Enter the product id:"))
        pquan=input("Enter the new product quantity:")
    except ValueError:
        print("Incorrect value entered!")
    else:
```

```
        query="Update product set p_qty=%s where p_id=%s"
        mycursor.execute(query, (pquan, id))
        mydb.commit()
        print("\nThe modified data is:")
        d={}
        j=1
        mycursor.execute("select * from product where p_id="+str(id))
        dt=["PRODUCT ID", "PRODUCT NAME", "PRODUCT COMPANY",
"PRODUCT PRICE(PER UNIT)", "QUANTITY", "DISCOUNT",
"PURCHASE DATE"]
        for i in mycursor:
            data = [i[0], i[1], i[2], i[3],i[4],i[5],i[6],i[7]]
            d[j]=data
            j+=1
        print ("{:<15} {:<15} {:<18} {:<22} {:<15} {:<15} {:<15}
{:<15}".format("PRODUCT ID", "PRODUCT NAME", "PRODUCT
COMPANY", "PRODUCT PRICE(PER UNIT)", "QUANTITY",
"DISCOUNT", "RATE", "PURCHASE DATE"))
        for k, v in d.items():
            a, b, c, h, e, f, g=v
            print("{:<15} {:<15} {:<18} {:<24} {:<18} {:<15} {:<15}
{:<15}".format(a, b, c, h, e, f, str(g)))



def prod_disc():
    try:
        id=int(input("Enter the product id:"))
        pdisc=input("Enter the new product discount:")
```

```
    except ValueError:
        print("Incorrect value entered!")
    else:
        query="Update product set p_discount=%s where p_id=%s"
        mycursor.execute(query, (pdisc, id))
        mydb.commit()
        print("\nThe modified data is:")
        d={}
        j=1
        mycursor.execute("select * from product where p_id="+str(id))
        dt=["PRODUCT ID", "PRODUCT NAME", "PRODUCT COMPANY",
"PRODUCT PRICE(PER UNIT)", "QUANTITY", "DISCOUNT",
"PURCHASE DATE"]
        for i in mycursor:
            data = [i[0], i[1], i[2], i[3],i[4],i[5],i[6],i[7]]
            d[j]=data
            j+=1
        print ("{:<15} {:<15} {:<18} {:<22} {:<15} {:<15} {:<15}
{:<15}".format("PRODUCT ID", "PRODUCT NAME", "PRODUCT
COMPANY", "PRODUCT PRICE(PER UNIT)", "QUANTITY",
"DISCOUNT", "RATE", "PURCHASE DATE"))
        for k, v in d.items():
            a, b, c, h, e, f, g=v
            print("{:<15} {:<15} {:<18} {:<24} {:<18} {:<15} {:<15}
{:<15}".format(a, b, c, h, e, f, str(g)))



def prod_purchase():
```

```
    try:
        id=int(input("Enter the product id:"))
        ppurchase=input("Enter the new product purchase date (yyyy-mm-dd):")
    except ValueError:
        print("\nIncorrect value entered!")
    else:
        query="Update product set p_purchasedate=%s where p_id=%s"
        mycursor.execute(query, (ppurchase, id))
        mydb.commit()
        print("The modified data is:")
        d={}
        j=1
        mycursor.execute("select * from product where p_id="+str(id))
        dt=["PRODUCT ID", "PRODUCT NAME", "PRODUCT COMPANY",
"PRODUCT PRICE(PER UNIT)", "QUANTITY", "DISCOUNT",
"PURCHASE DATE"]
        for i in mycursor:
            data = [i[0], i[1], i[2], i[3],i[4],i[5],i[6],i[7]]
            d[j]=data
            j+=1
        print ("{:<15} {:<15} {:<18} {:<22} {:<15} {:<15} {:<15}
{:<15}".format("PRODUCT ID", "PRODUCT NAME", "PRODUCT
COMPANY", "PRODUCT PRICE(PER UNIT)", "QUANTITY",
"DISCOUNT", "RATE", "PURCHASE DATE"))
        for k, v in d.items():
            a, b, c, h, e, f, g=v
            print("{:<15} {:<15} {:<18} {:<24} {:<18} {:<15} {:<15}
{:<15}".format(a, b, c, h, e, f, str(g)))
```

```
def delete_rec():
    try:
        id=int(input("Enter the product id:"))
    except ValueError:
        print("Incorrect value entered!")
    else:
        mycursor.execute("DELETE from product where p_id="+str(id))
        print("\nThe modified data is:")
        d={}
        j=1
        mycursor.execute("select * from product")
        dt=["PRODUCT ID", "PRODUCT NAME", "PRODUCT COMPANY",
"PRODUCT PRICE(PER UNIT)", "QUANTITY", "DISCOUNT",
"PURCHASE DATE"]
        for i in mycursor:
            data = [i[0], i[1], i[2], i[3],i[4],i[5],i[6],i[7]]
            d[j]=data
            j+=1
        print ("{:<15} {:<15} {:<18} {:<22} {:<15} {:<15} {:<15}
{:<15}".format("PRODUCT ID", "PRODUCT NAME", "PRODUCT
COMPANY", "PRODUCT PRICE(PER UNIT)", "QUANTITY",
"DISCOUNT", "RATE", "PURCHASE DATE"))
        for k, v in d.items():
            a, b, c, h, e, f, g=v
            print("{:<15} {:<15} {:<18} {:<24} {:<18} {:<15} {:<15}
{:<15}".format(a, b, c, h, e, f, str(g)))
```

```
def prod_menu():
    ch=0
    while ch!=6:
        print("="*30+"PRODUCT MENU"+"="*30)
        print("1. CREATE PRODUCT")
        print("2. DISPLAY ALL PRODUCTS AVAILABLE")
        print("3. SEARCH DESIRED RECORD")
        print("4. MODIFY PRODUCT RECORD")
        print("5. DELETE PRODUCT RECORD")
        print("6. BACK TO ADMINISTRATOR MENU")
        print("="*73)
        ch=int(input("Enter your choice (1-7):"))
        if ch==1:
            create_rec()
        elif ch==2:
            display_rec()
        elif ch==3:
            search_rec()
        elif ch==4:
            modify_rec()
        elif ch==5:
            delete_rec()
        elif ch==6:
```

```
            exit
        else:
            print("Incorrect choice! Please enter again!")



#CUSTOMER MENU STARTS ----------------------------------------------------
-------------------------------------------------------------------------------------------
----------------------------------------------------
def create_record():
    while True:
        try:
            custid=int(input("Enter CUSTOMER ID: "))
            custname=input("Enter CUSTOMER NAME: ")
            address=input("Enter CUSTOMER ADDRESS: ")
            phone=input("Enter CUSTOMER PHONE NO.: ")
        except ValueError:
            print("Incorrect value entered!")
        else:
            data=(custid, custname, address, phone)
            query='insert into customer values(%s,%s,%s,%s)'
            mycursor.execute(query, data)
            mydb.commit()
            more=input('Do you want to add more records? (y/n): ')
            if more in 'nN':
                break
            print('\nRecord added!')
```

```
def display_record():
    d={}
    j=1
    try:
        mycursor.execute('select * from customer')
        print()
        dt=['CUSTOMER ID', 'CUSTOMER NAME', 'CUSTOMER ADDRESS',
'PHONE NO.']
        for i in mycursor:
            data=[i[0], i[1], i[2], i[3]]
            d[j]=data
            j+=1
        print('{:<15} {:<22} {:<44} {:<15}'.format('CUSTOMER ID',
'CUSTOMER NAME', 'CUSTOMER ADDRESS', 'PHONE NO.'))
        for k, v in d.items():
            a, b, c, e=v
            print('{:<15} {:<22} {:<44} {:<15}'.format(a, b, c, e))
    except:
        print("Error encountered in displaying record from table!")


def search_record():
    d={}
    j=1
    try:
        key=input("Enter Customer ID to search for the record: ")
        query='select * from customer where cust_id=%s'
        mycursor.execute(query, (key,))
```

```
        print()
        dt=['CUSTOMER ID', 'CUSTOMER NAME', 'CUSTOMER ADDRESS',
'PHONE NO.']
        for i in mycursor:
            data=[i[0], i[1], i[2], i[3]]
            d[j]=data
            j+=1
        print('{:<15} {:<22} {:<44} {:<15}'.format('CUSTOMER ID',
'CUSTOMER NAME', 'CUSTOMER ADDRESS', 'PHONE NO.'))
        for k, v in d.items():
            a, b, c, e=v
            print('{:<15} {:<22} {:<44} {:<15}'.format(a, b, c, e))
    except:
         print("Error encountered in displaying record from table!")


def modify_record():
    while True:
        print('/N'+'='*30+'CUSTOMER RECORD MODIFICATION'+'='*30)
        print('1. MODIFY CUSTOMER NAME')
        print('2. MODIFY CUSTOMER ADDRESS')
        print('3. MODIFY CUSTOMER PHONE NO.')
        print('0. BACK TO CUSTOMER MENU')
        ch=input('\nEnter the choice to proceed (1/2/3/0): ')
        if ch=='1':
            cust_name()
        elif ch=='2':
            cust_address()
```

```
        elif ch=='3':
            cust_phone()
        elif ch=='0':
            break
        else:
            print('Invalid choice! Please enter again!')

def cust_name():
    try:
        cid=input('\nEnter Customer ID for the record to be modified: ')
        cname=input('Enter the New Customer name: ')
    except ValueError:
        print("Incorrect value entered!")
    else:
        query='update customer set cust_name=%s where cust_id=%s'
        data=(cname, cid)
        mycursor.execute(query, data)
        print('\nThe modified data is:')
        d={}
        j=1
        mycursor.execute('select * from customer where cust_id='+cid)
        for i in mycursor:
            data=[i[0], i[1], i[2], i[3]]
            d[j]=data
            j+=1
```

```
        print('{:<15} {:<22} {:<44} {:<15}'.format('CUSTOMER ID',
'CUSTOMER NAME', 'CUSTOMER ADDRESS', 'PHONE NO.'))
        for k, v in d.items():
            a, b, c, e=v
            print('{:<15} {:<22} {:<44} {:<15}'.format(a, b, c, e))


def cust_address():
    try:
        cid=input('\nEnter Customer ID for the record to be modified: ')
        add=input('Enter the Updated Address: ')
    except ValueError:
        print("Incorrect value entered!")
    else:
        query='update customer set address=%s where cust_id=%s'
        data=(add, cid)
        mycursor.execute(query, data)
        print('\nThe modified data is:')
        d={}
        j=1
        mycursor.execute('select * from customer where cust_id='+cid)
        for i in mycursor:
            data=[i[0], i[1], i[2], i[3]]
            d[j]=data
            j+=1
        print('{:<15} {:<22} {:<44} {:<15}'.format('CUSTOMER ID',
'CUSTOMER NAME', 'CUSTOMER ADDRESS', 'PHONE NO.'))
        for k, v in d.items():
```

```
            a, b, c, e=v
            print('{:<15} {:<22} {:<44} {:<15}'.format(a, b, c, e))


def cust_phone():
    try:
        cid=input('\nEnter Customer ID for the record to be modified: ')
        phn=input('Enter the Updated Phone no.: ')
    except ValueError:
        print("Incorrect value entered")
    else:
        query='update customer set phone=%s where cust_id=%s'
        data=(phn, cid)
        mycursor.execute(query, data)
        print('\nThe modified data is:')
        d={}
        j=1
        mycursor.execute('select * from customer where cust_id='+cid)
        for i in mycursor:
            data=[i[0], i[1], i[2], i[3]]
            d[j]=data
            j+=1
        print('{:<15} {:<22} {:<44} {:<15}'.format('CUSTOMER ID',
'CUSTOMER NAME', 'CUSTOMER ADDRESS', 'PHONE NO.'))
        for k, v in d.items():
            a, b, c, e=v
            print('{:<15} {:<22} {:<44} {:<15}'.format(a, b, c, e))
```

```
def delete_record():
    try:
        cid=input('\nEnter Customer ID for the record to be deleted: ')
    except ValueError:
        print("Incorrect value entered!")
    else:
        query='delete from customer where cust_id=%s'
        mycursor.execute(query, (cid,))
        print('\nThe updated data is:')
        d={}
        j=1
        mycursor.execute('select * from customer')
        for i in mycursor:
            data=[i[0], i[1], i[2], i[3]]
            d[j]=data
            j+=1
        print('{:<15} {:<22} {:<44} {:<15}'.format('CUSTOMER ID',
'CUSTOMER NAME', 'CUSTOMER ADDRESS', 'PHONE NO.'))
        for k, v in d.items():
            a, b, c, e=v
            print('{:<15} {:<22} {:<44} {:<15}'.format(a, b, c, e))


def cust_menu():
    while True:
        print()
```

```
print('='*30+'CUSTOMER MENU'+'='*30)
print('\n1. Create record')
print('2. Display record')
print('3. Search record')
print('4. Modify record')
print('5. Delete record')
print('0. Back to administrator menu')
print(25*'*  ')
ch=input('\nEnter your choice (1/2/3/4/5/0): ')
if ch=='1':
    create_record()
elif ch=='2':
    display_record()
elif ch=='3':
    search_record()
elif ch=='4':
    modify_record()
elif ch=='5':
    delete_record()
elif ch=='0':
    break
else:
    print("Incorrect choice! Please enter again!")
```

```
#ADMIN MENU STARTS----------------------------------------------------------------
--------------------------------------------------------------------------------------
-----------------------------------------------------
def admin():
    while True:
        print("="*30+"ADMINISTRATOR MENU"+"="*30)
        print("1. CUSTOMERS MENU")
        print("2. PRODUCTS MENU")
        print("3. CANTEEN SALE MENU")
        print("4. BACK TO MAIN MENU")
        print("="*73)
        ch=int(input("Enter your choice (1-7):"))
        if ch==1:
            cust_menu()
        elif ch==2:
            prod_menu()
        elif ch==3:
            cant_sales()
        elif ch==4:
            exit
        else:
            print("Incorrect choice! Please enter again!")



#PRODUCT REPORT GENERATOR STARTS---------------------------------------
--------------------------------------------------------------------------------------
-------------------------------------------------------------
```

```
def prod_report():
    d={}
    j=1
    try:
        mycursor.execute("select * from product_report")
        dt=["PRODUCT ID", "PURCHASE DATE", "PRODUCT PAYMENT
STATUS", "DUE AMOUNT"]
        for i in mycursor:
            data = [i[0], i[1], i[2], i[3]]
            d[j]=data
            j+=1
        print ("{:<15} {:<18} {:<18} {:<22}".format("PRODUCT ID",
"PURCHASE DATE", "PAYMENT STATUS", "DUE AMOUNT"))
        for k, v in d.items():
            a, b, c, h =v
            print("{:<15} {:<18} {:<18} {:<22}".format(a, str(b), c, h))
    except:
        print("Error encountered in displaying record from table!")



#MAIN MENU STARTS----------------------------------------------------------------
--------------------------------------------------------------------------------------------
----------------------------------------------------


def main_menu():
    while True:
```

```
        print("="*30+"MAIN MENU"+"="*30)
        print("1. PRODUCTS REPORT GENERATOR")
        print("2. ADMINISTRATOR")
        print("3. EXIT")
        print("="*73)
        ch=int(input("Enter your choice (1-3):"))
        if ch==1:
            prod_report()
        elif ch==2:
            admin()
        elif ch==3:
            exit
        else:
            print("Incorrect choice! Please enter again!")
#main_menu()


#CANTEEN SALES MENU STARTS----------------------------------------------
-------------------------------------------------------------------------------------------
----------------------------------------------------

def add_purchase():
    while True:
        try:
            c_id=int(input('Enter CUSTOMER ID: '))
            p_id=int(input('Enter PRODUCT ID: '))
            qty=int(input('Enter QUANTITY PURCHASED: '))
```

```
        except ValueError:
            print('Incorrect value entered!')
        else:
            query1='insert into purchase select cust_id, cust_name, p_id, p_name,
p_company, rate, {0}, rate*{0} as amount, curdate() from customer natural join
product where cust_id={1} and p_id={2}'.format(qty, c_id, p_id)
            mycursor.execute(query1)
            query2='update product set p_qty=p_qty-%s where p_id=%s' % (qty,
p_id)
            mycursor.execute(query2)
            mydb.commit()
            ch=input('Do you want to add more records? (y/n): ')
            if ch in 'nN':
                break
            print('\nRecord added!')


def del_purchase():
    try:
        c_id=int(input('Enter CUSTOMER ID: '))
        p_id=int(input('Enter PRODUCT ID: '))
        date=input('Enter DATE OF PURCHASE (yyyy-mm-dd): ')
    except ValueError:
        print("Incorrect value entered!")
    else:
        query='delete from purchase where cust_id=%s and p_id=%s and
sale_date=%s'
        mycursor.execute(query, (c_id, p_id, date))
```

```
        mydb.commit()
        print('\nThe updated data is:')
        d={}
        j=1
        mycursor.execute('select * from purchase')
        for i in mycursor:
            data=[i[0], i[1], i[2], i[3], i[4], i[5], i[6], i[7], i[8]]
            d[j]=data
            j+=1
        print('{:<13} {:<20} {:<13} {:<20} {:<20} {:<10} {:<10} {:<10}
{:<15}'.format('CUSTOMER ID', 'CUSTOMER NAME', 'PRODUCT ID',
'PRODUCT NAME', 'PRODUCT COMPANY', 'RATE', 'QUANTITY',
'AMOUNT', 'DATE'))
        for k, v in d.items():
            a, b, c, e, f, g, h, l, m=v
            print('{:<13} {:<20} {:<13} {:<20} {:<20} {:<10} {:<10} {:<10}
{:<15}'.format(a, b, c, e, f, g, h, l, m))



def purch_qty():
    try:
        c_id=int(input('Enter CUSTOMER ID: '))
        p_id=int(input('Enter PRODUCT ID: '))
        qty=int(input('Enter UPDATED QUANTITY: '))
        date=input('Enter DATE OF PURCHASE (yyyy-mm-dd): ')
    except ValueError:
        print("Incorrect value entered!")
```

```
    else:
        #update qty in purchase table
        data=(qty, c_id, p_id, date)
        query='update purchase set quantity=%s where c_id=%s and p_id=%s and
date=%s'
        mycursor.execute(query, data)
        mydb.commit()


        print('\nThe updated data is:')
        d={}
        j=1
        mycursor.execute('select * from purchase')
        for i in mycursor:
            data=[i[0], i[1], i[2], i[3], i[4], i[5], i[6], i[7], i[8]]
            d[j]=data
            j+=1
        print('{:<15} {:<22} {:<15} {:<20} {:<20} {:<10} {:<10} {:<10}
{:<15}'.format('CUSTOMER ID', 'CUSTOMER NAME', 'PRODUCT ID',
'PRODUCT NAME', 'PRODUCT COMPANY', 'RATE', 'QUANTITY',
'AMOUNT', 'DATE'))
        for k, v in d.items():
            a, b, c, e, f, g, h, l, m=v
            print('{:<15} {:<22} {:<15} {:<20} {:<20} {:<10} {:<10} {:<10}
{:<15}'.format(a, b, c, e, f, g, h, l, m))


        #update qty in product table
        mycursor.execute('select quantity from purchase where p_id='+str(p_id)+'
and cust_id='+str(c_id)+' and date='+str(date))
```

```
        quan=mycursor.fetchone()[0]
        data1=(quan-qty, c_id, p_id, date)
        query1='update product set p_qty=p_qty+%s where c_id=%s and p_id=%s
and date=%s'
        mycursor.execute(query1, data1)
        mydb.commit()



def cant_sales():
    while True:
        print(25*'='+'CANTEEN SALES MENU'+25*'=')
        print('\n1. ADD CUSTOMER PURCHASE')
        print('2. MODIFY PURCHASE QUANTITY')
        print('3. DELETE CUSTOMER PURCHASE')
        print('0. BACK TO ADMIN MENU')
        print(73*'=')
        ch=input('Enter your choice (1/2/3/0): ')
        if ch=='1':
            add_purchase()
        elif ch=='2':
            purch_qty()
        elif ch=='3':
            del_purchase()
        elif ch=='0':
            break
        else:
```

```
            print("Incorrect choice! Please enter again!")



#CUSTOMER PRODUCT MENU STARTS----------------------------------------
--------------------------------------------------------------------------------------------
------------------------------------------------------------
def cust_prod():
    while True:
        print(25*'='+'CUSTOMER PRODUCT MENU'+25*'=')
        print('\n1. CUSTOMER MENU')
        print('2. PRODUCT MENU')
        print('3. CANTEEN SALES MENU')
        print('0. BACK TO ADMIN MENU')
        print(73*'=')
        ch=input('Enter your choice (1/2/3/0): ')
        if ch=='1':
            cust_menu()
        elif ch=='2':
            prod_menu()
        elif ch=='3':
            cant_sales()
        elif ch=='0':
            break
        else:
            print("Incorrect choice! Please enter again!")
```

```
#INVOICE GENERATOR-----------------------------------------------------------------
--------------------------------------------------------------------------------
def invoice():
    while True:
        cust_id=int(input('\nEnter CUSTOMER ID: '))
        date=input('Enter DATE OF PURCHASE (yyyy-mm-dd): ')
        print()
        print('INVOICE'.center(100))
        #print(f"{'INVOICE':^100}")
        cust='select cust_name, phone from customer where cust_id=%s' %
(cust_id,)
        mycursor.execute(cust)
        l=[]
        for i in mycursor:
            for j in i:
                l.append(j)
        print('Customer ID :', cust_id, f"{'Date : '+date :^100}")
        print('Customer Name :', l[0])
        print('Customer Phone No. :', l[1])
        print()
        d={}
        j=1
        query='select p_id, p_name, p_company, rate, quantity, amount from
purchase where cust_id=%s and sale_date=%s'
        data=(cust_id, date)
        mycursor.execute(query, data)
        for i in mycursor:
```

```
            data=[i[0], i[1], i[2], i[3], i[4], i[5]]
            d[j]=data
            j+=1
        print('{:<13} {:<20} {:<20} {:<10} {:<10} {:<10}'.format('PRODUCT
ID', 'PRODUCT NAME', 'PRODUCT COMPANY', 'RATE', 'QUANTITY',
'AMOUNT'))
        for k, v in d.items():
            a, b, c, e, f, g=v
            print('{:<13} {:<20} {:<20} {:<10} {:<10} {:<10} '.format(a, b, c, e, f,
g))

        q='select sum(amount) from purchase where cust_id=%s and
sale_date=%s'
        mycursor.execute(q, (cust_id, date))
        total=mycursor.fetchone()[0]

        print('{:<13} {:<20} {:<20} {:<10} {:<10}
{:<10}'.format(",",",",'TOTAL',total))

        ch=input('\nWant to generate another invoice? (y/n): ')
        if ch in 'nN':
            break

invoice()
```

TESTING:

The system was tested with different use cases, such as purchasing the stock and maintaining its record, sales of the canteen, and updating the database.

Unit Testing: I tested individual functions (like create_rec and add_purchase) separately to ensure they perform their specific task without errors.

Validation Testing: I intentionally entered incorrect data types (e.g., text instead of numbers) to verify that the system's error handling (tryexcept blocks) works and prevents runtime crashes.

Integration Testing: I specifically tested the "Sales" module to confirm that buying a product automatically reduces the stock count in the "Product" inventory table, ensuring the two modules interact correctly.

OUTPUT:

==========MAIN MENU==========

1. PRODUCTS REPORT GENERATOR
2. CUSTOMER INVOICE GENERATOR
3. ADMINISTRATOR
4. EXIT

==========

Enter your choice (1-4): 3

==========ADMINISTRATOR MENU==========

1. CUSTOMERS MENU
2. PRODUCTS MENU
3. CANTEEN SALE MENU
4. BACK TO MAIN MENU

==========

Enter your choice (1-4): 1

==========CUSTOMER MENU==========

1. Create record
2. Display record
3. Search record
4. Modify record
5. Delete record
0. Back to administrator menu

==========

Enter your choice (1/2/3/4/5/0): 1

Enter CUSTOMER ID: 100001

Enter CUSTOMER NAME: Rakhi Sharma

Enter CUSTOMER ADDRESS: B/102, ALF Apartments, MD Road

Enter CUSTOMER PHONE NO.: 9910435690

Do you want to add more records? (y/n): y

Enter CUSTOMER ID: 120034
Enter CUSTOMER NAME: Amar Saini
Enter CUSTOMER ADDRESS: A/12, Defence Colony
Enter CUSTOMER PHONE NO.: 7705704832

Do you want to add more records? (y/n): y

Enter CUSTOMER ID: 123456
Enter CUSTOMER NAME: Kushagra Patel
Enter CUSTOMER ADDRESS: 536, IFFCO Colony
Enter CUSTOMER PHONE NO.: 8800123654

Do you want to add more records? (y/n): y

Enter CUSTOMER ID: 110045
Enter CUSTOMER NAME: Avantika Mishra
Enter CUSTOMER ADDRESS: 1860, Shivaji Nagar
Enter CUSTOMER PHONE NO.: 9872046300

Do you want to add more records? (y/n): y

Enter CUSTOMER ID: 132008
Enter CUSTOMER NAME: Ashok Sinha
Enter CUSTOMER ADDRESS: C/82, Ekta Apartments, Sector 15
Enter CUSTOMER PHONE NO.: 8240963537

Do you want to add more records? (y/n): y

Enter CUSTOMER ID: 145879

Enter CUSTOMER NAME: Ishita Singh

Enter CUSTOMER ADDRESS: 342, Shankar Vihar

Enter CUSTOMER PHONE NO.: 9344552687

Do you want to add more records? (y/n): n

Record(s) added!

==========CUSTOMER MENU==========

1. CREATE RECORD
2. DISPLAY RECORD
3. SEARCH RECORD
4. MODIFY RECORD
5. DELETE RECORD
0. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1/2/3/4/5/0): 2

CUSTOMER ID	CUSTOMER NAME	CUSTOMER ADDRESS	PHONE NO.
100001	Rakhi Sharma	B/102, ALF Apartments, MD Road	9910435690
110045	Avantika Mishra	1860, Shivaji Nagar	9872046300
120034	Amar Saini	A/12, Defence Colony	7705704832
123456	Kushagra Patel	536, IFFCO Colony	8800123654
132008	Ashok Sinha	C/82, Ekta Apartments, Sector 15	8240963537
145879	Ishita Singh	342, Shankar Vihar	9344552687

==========CUSTOMER MENU==========

1. CREATE RECORD
2. DISPLAY RECORD
3. SEARCH RECORD
4. MODIFY RECORD
5. DELETE RECORD

0. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1/2/3/4/5/0): 3

Enter Customer ID to search for the record: 120034

CUSTOMER ID	CUSTOMER NAME	CUSTOMER ADDRESS	PHONE NO.
120034	Amar Saini	A/12, Defence Colony	7705704832

==========CUSTOMER MENU==========

1. CREATE RECORD
2. DISPLAY RECORD
3. SEARCH RECORD
4. MODIFY RECORD
5. DELETE RECORD
0. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1/2/3/4/5/0): 4

==========CUSTOMER RECORD MODIFICATION==========

1. MODIFY CUSTOMER NAME
2. MODIFY CUSTOMER ADDRESS
3. MODIFY CUSTOMER PHONE NO.
0. BACK TO CUSTOMER MENU

Enter the choice to proceed (1/2/3/0): 3

Enter Customer ID for the record to be modified: 123456

Enter the Updated Phone no.: 8970125647

The modified data is:

CUSTOMER ID	CUSTOMER NAME	CUSTOMER ADDRESS	PHONE NO.

123456 Kushagra Patel 536, IFFCO Colony 8970125647

==========CUSTOMER RECORD MODIFICATION==========

1. MODIFY CUSTOMER NAME
2. MODIFY CUSTOMER ADDRESS
3. MODIFY CUSTOMER PHONE NO.
0. BACK TO CUSTOMER MENU

Enter the choice to proceed (1/2/3/0): 0

==========CUSTOMER MENU==========

1. CREATE RECORD
2. DISPLAY RECORD
3. SEARCH RECORD
4. MODIFY RECORD
5. DELETE RECORD
0. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1/2/3/4/5/0): 5

Enter Customer ID for the record to be deleted: 145879

The updated data is:

CUSTOMER ID	CUSTOMER NAME	CUSTOMER ADDRESS	PHONE NO.
100001	Rakhi Sharma	B/102, ALF Apartments, MD Road	9910435690
110045	Avantika Mishra	1860, Shivaji Nagar	9872046300
120034	Amar Saini	A/12, Defence Colony	7705704832
123456	Kushagra Patel	536, IFFCO Colony	8970125647
132008	Ashok Sinha	C/82, Ekta Apartments, Sector 15	8240963537

==========CUSTOMER MENU==========

1. CREATE RECORD
2. DISPLAY RECORD

3. SEARCH RECORD

4. MODIFY RECORD

5. DELETE RECORD

0. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1/2/3/4/5/0): 0

==========ADMINISTRATOR MENU==========

1. CUSTOMERS MENU

2. PRODUCTS MENU

3. CANTEEN SALE MENU

4. BACK TO MAIN MENU

==========

Enter your choice (1-4): 2

==========PRODUCT MENU==========

1. CREATE PRODUCT

2. DISPLAY ALL PRODUCTS AVAILABLE

3. SEARCH DESIRED RECORD

4. MODIFY PRODUCT RECORD

5. DELETE PRODUCT RECORD

6. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1-7): 1

PRODUCT ID: 101

NAME OF PRODUCT: Chocolate

NAME OF PRODUCT COMPANY: Cadbury

PER UNIT PRODUCT PRICE: 45

PRODUCT QUANTITY: 200

PRODUCT DISCOUNT: 100

PRODUCT SELLING RATE: 50

PRODUCT PURCHASE DATE (yyyy-mm-dd): 2024-01-03
Is the outstanding payment made? (Yes/No): Yes

Do you want to enter more data?(y/n): y

PRODUCT ID: 102
NAME OF PRODUCT: Coffee Powder
NAME OF PRODUCT COMPANY: Nescafe
PER UNIT PRODUCT PRICE: 20
PRODUCT QUANTITY: 300
PRODUCT DISCOUNT: 80
PRODUCT SELLING RATE: 30
PRODUCT PURCHASE DATE (yyyy-mm-dd): 2024-04-01
Is the outstanding payment made? (Yes/No): No
Enter the due amount for product purchase: 3000

Do you want to enter more data?(y/n): y

PRODUCT ID: 103
NAME OF PRODUCT: Toothpaste
NAME OF PRODUCT COMPANY: Colgate
PER UNIT PRODUCT PRICE: 86
PRODUCT QUANTITY: 100
PRODUCT DISCOUNT: 0
PRODUCT SELLING RATE: 90
PRODUCT PURCHASE DATE (yyyy-mm-dd): 2024-03-01
Is the outstanding payment made? (Yes/No): Yes

Do you want to enter more data?(y/n): y

PRODUCT ID: 104
NAME OF PRODUCT: Tea Leaves

NAME OF PRODUCT COMPANY: Red Label

PER UNIT PRODUCT PRICE: 70

PRODUCT QUANTITY: 400

PRODUCT DISCOUNT: 100

PRODUCT SELLING RATE: 80

PRODUCT PURCHASE DATE (yyyy-mm-dd): 2024-02-11

Is the outstanding payment made? (Yes/No): Yes

Do you want to enter more data?(y/n): y

PRODUCT ID: 105

NAME OF PRODUCT: Soap

NAME OF PRODUCT COMPANY: Pears

PER UNIT PRODUCT PRICE: 30

PRODUCT QUANTITY: 200

PRODUCT DISCOUNT: 100

PRODUCT SELLING RATE: 25

PRODUCT PURCHASE DATE (yyyy-mm-dd): 2024-04-03

Is the outstanding payment made? (Yes/No): No

Enter the due amount for product purchase: 4000

Do you want to enter more data?(y/n): y

PRODUCT ID: 106

NAME OF PRODUCT: Biscuit

NAME OF PRODUCT COMPANY: Parle G

PER UNIT PRODUCT PRICE: 10

PRODUCT QUANTITY: 500

PRODUCT DISCOUNT: 150

PRODUCT SELLING RATE: 15

PRODUCT PURCHASE DATE (yyyy-mm-dd): 2024-05-10

Is the outstanding payment made? (Yes/No): Yes

Do you want to enter more data?(y/n): n

Record(s) added!

==========PRODUCT MENU==========

1. CREATE PRODUCT
2. DISPLAY ALL PRODUCTS AVAILABLE
3. SEARCH DESIRED RECORD
4. MODIFY PRODUCT RECORD
5. DELETE PRODUCT RECORD
6. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1-7): 2

PRODUCT ID PRODUCT NAME PRODUCT COMPANY PRODUCT PRICE(PER UNIT) QUANTITY DISCOUNT RATE PURCHASE DATE

101 Chocolate Cadbury 45.0 200 100.0 50.0 2024-01-03

102 Coffee Powder Nescafe 20.0 300 80.0 30.0 2024-04-01

103 Toothpaste Colgate 86.0 100 0.0 90.0 2024-03-01

104 Tea Leaves Red Label 70.0 400 100.0 80.0 2024-02-11

105 Soap Pears 30.0 200 100.0 25.0 2024-04-03

106 Biscuit Parle G 10.0 500 150.0 15.0 2024-05-10

==========PRODUCT MENU==========

1. CREATE PRODUCT
2. DISPLAY ALL PRODUCTS AVAILABLE
3. SEARCH DESIRED RECORD
4. MODIFY PRODUCT RECORD

5. DELETE PRODUCT RECORD

6. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1-7): 3

Enter the product id:103

PRODUCT ID PRODUCT NAME PRODUCT COMPANY PRODUCT PRICE(PER UNIT) QUANTITY DISCOUNT RATE PURCHASE DATE

103 Toothpaste Colgate 86.0 100 0.0 90.0 2024-03-01

==========PRODUCT MENU==========

1. CREATE PRODUCT

2. DISPLAY ALL PRODUCTS AVAILABLE

3. SEARCH DESIRED RECORD

4. MODIFY PRODUCT RECORD

5. DELETE PRODUCT RECORD

6. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1-7):4

==========MODIFY PRODUCT-RECORD MENU==========

1. MODIFY PRODUCT NAME

2. MODIFY PRODUCT COMPANY

3. MODIFY PRODUCT PRICE

4. MODIFY PRODUCT QUANTITY

5. MODIFY PRODUCT DISCOUNT

6. MODIFY PRODUCT SELLING RATE

7. MODIFY PRODUCT PURCHASE DATE

8. BACK TO PRODUCT MENU

==========

Enter your choice (1-8):6

Enter the product id:101

Enter the new product rate:40

The modified data is:

PRODUCT ID	PRODUCT NAME	PRODUCT COMPANY	PRODUCT PRICE(PER UNIT)	QUANTITY	DISCOUNT	RATE	PURCHASE DATE
101	Chocolate	Cadbury	45.0	200	100.0	40.0	2024-01-03

==========MODIFY PRODUCT-RECORD MENU==========

1. MODIFY PRODUCT NAME
2. MODIFY PRODUCT COMPANY
3. MODIFY PRODUCT PRICE
4. MODIFY PRODUCT QUANTITY
5. MODIFY PRODUCT DISCOUNT
6. MODIFY PRODUCT SELLING RATE
7. MODIFY PRODUCT PURCHASE DATE
8. BACK TO PRODUCT MENU

==========

Enter your choice (1-8):8

==========PRODUCT MENU==========

1. CREATE PRODUCT
2. DISPLAY ALL PRODUCTS AVAILABLE
3. SEARCH DESIRED RECORD
4. MODIFY PRODUCT RECORD
5. DELETE PRODUCT RECORD
6. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1-7):5

Enter the product id:103

The modified data is:

PRODUCT ID PRODUCT NAME PRODUCT COMPANY PRODUCT PRICE(PER UNIT) QUANTITY DISCOUNT RATE PURCHASE DATE

101 Chocolate Cadbury 45.0 200 100.0 40.0 2024-01-03

102 Coffee Powder Nescafe 20.0 300 80.0 30.0 2024-04-01

104 Tea Leaves Red Label 70.0 400 100.0 80.0 2024-02-11

105 Soap Pears 30.0 200 100.0 25.0 2024-04-03

106 Biscuit Parle G 10.0 500 150.0 15.0 2024-05-10

==========PRODUCT MENU==========

1. CREATE PRODUCT
2. DISPLAY ALL PRODUCTS AVAILABLE
3. SEARCH DESIRED RECORD
4. MODIFY PRODUCT RECORD
5. DELETE PRODUCT RECORD
6. BACK TO ADMINISTRATOR MENU

==========

Enter your choice (1-7):6

==========ADMINISTRATOR MENU==========

1. CUSTOMERS MENU
2. PRODUCTS MENU
3. CANTEEN SALE MENU
4. BACK TO MAIN MENU

==========

Enter your choice (1-4): 4

==========MAIN MENU==========

1. PRODUCTS REPORT GENERATOR
2. CUSTOMER INVOICE GENERATOR

3. ADMINISTRATOR

4. EXIT

==========

Enter your choice (1-4):1

PRODUCT ID	PURCHASE DATE	PAYMENT STATUS	DUE AMOUNT
101	2024-01-03	Yes	0.0
102	2024-04-01	No	3000.0
103	2024-03-01	Yes	0.0
104	2024-02-11	Yes	0.0
105	2024-04-03	No	4000.0
106	2024-05-10	Yes	0.0

==========MAIN MENU==========

1. PRODUCTS REPORT GENERATOR

2. CUSTOMER INVOICE GENERATOR

3. ADMINISTRATOR

4. EXIT

==========

Enter your choice (1-4):3

==========ADMINISTRATOR MENU==========

1. CUSTOMERS MENU

2. PRODUCTS MENU

3. CANTEEN SALE MENU

4. BACK TO MAIN MENU

==========

Enter your choice (1-4):3

==========CANTEEN SALES MENU==========

1. ADD CANTEEN SALE

2. MODIFY SALE QUANTITY

3. DELETE CANTEEN SALE

0. BACK TO ADMIN MENU

==========

Enter your choice (1/2/3/0): 1

Enter CUSTOMER ID: 100001

Enter PRODUCT ID: 102

Enter QUANTITY SOLD: 5

Do you want to add more records? (y/n): y

Enter CUSTOMER ID: 123456

Enter PRODUCT ID: 105

Enter QUANTITY SOLD: 2

Do you want to add more records? (y/n): n

Record(s) added!

==========CANTEEN SALES MENU==========

1. ADD CANTEEN SALE

2. MODIFY SALE QUANTITY

3. DELETE CANTEEN SALE

0. BACK TO ADMIN MENU

==========

Enter your choice (1/2/3/0): 2

Enter CUSTOMER ID: 100001

Enter PRODUCT ID: 102

Enter UPDATED QUANTITY: 3

Enter DATE OF SALE (yyyy-mm-dd): 2024-11-05

The updated data is:

CUSTOMER ID CUSTOMER NAME PRODUCT ID PRODUCT NAME PRODUCT COMPANY RATE QUANTITY AMOUNT DATE

100001 Rakhi Sharma 102 Coffee Powder Nescafe 30.0 3 90.0 2024-11-05

123456 Kushagra Patel 105 Soap Pears 25.0 2 50.0 2024-11-05

120034 Amar Saini 104 Tea Leaves Red Label 80.0 1 80.0 2024-09-15

132008 Ashok Sinha 104 Tea Leaves Red Label 80.0 1 80.0 2024-10-25

132008 Ashok Sinha 105 Soap Pears 25.0 4 100.0 2024-10-25

132008 Ashok Sinha 106 Biscuit Parle G 15.0 10 150.0 2024-10-25

==========CANTEEN SALES MENU==========

1. ADD CANTEEN SALE

2. MODIFY SALE QUANTITY

3. DELETE CANTEEN SALE

0. BACK TO ADMIN MENU

==========

Enter your choice (1/2/3/0): 3

Enter CUSTOMER ID: 123456

Enter PRODUCT ID: 105

Enter DATE OF SALE (yyyy-mm-dd): 2024-11-05

The updated data is:

CUSTOMER ID CUSTOMER NAME PRODUCT ID PRODUCT NAME PRODUCT COMPANY RATE QUANTITY AMOUNT DATE

100001 Rakhi Sharma 102 Coffee Powder Nescafe 30.0 3 150.0 2024-11-05

120034 Amar Saini 104 Tea Leaves Red Label 80.0 1 80.0 2024-11-05

132008 Ashok Sinha 104 Tea Leaves Red Label 80.0 1 80.0 2024-10-25

132008 Ashok Sinha 105 Soap Pears 25.0 4
100.0 2024-10-25
132008 Ashok Sinha 106 Biscuit Parle G 15.0 10
150.0 2024-10-25

==========CANTEEN SALES MENU==========

1. ADD CANTEEN SALE
2. MODIFY SALE QUANTITY
3. DELETE CANTEEN SALE
0. BACK TO ADMIN MENU

==========

Enter your choice (1/2/3/0): 0

==========ADMINISTRATOR MENU==========

1. CUSTOMERS MENU
2. PRODUCTS MENU
3. CANTEEN SALE MENU
4. BACK TO MAIN MENU

==========

Enter your choice (1-4):4

==========MAIN MENU==========

1. PRODUCTS REPORT GENERATOR
2. CUSTOMER INVOICE GENERATOR
3. ADMINISTRATOR
4. EXIT

==========

Enter your choice (1-4):2

Enter CUSTOMER ID: 132008
Enter DATE OF SALE (yyyy-mm-dd): 2024-10-25

INVOICE

Customer ID : 132008 Date : 2024-10-25
Customer Name : Ashok Sinha

Customer Phone No. : 8240963537

PRODUCTID	PRODUCTNAME	PRODUCTCOMPANY	RATE	QUANTITY	AMOUNT
104	Tea Leaves	Red Label	80.0	1	80.0
105	Soap	Pears	25.0	4	100.0
106	Biscuit	Parle G	15.0	10	150.0
				TOTAL	330.0

Total GST: 5%

Amount after GST: Rs. 346.5

Payable Amount: Rs. 346.5

Want to generate another invoice? (y/n): n

==========MAIN MENU==========

1. PRODUCTS REPORT GENERATOR
2. CUSTOMER INVOICE GENERATOR
3. ADMINISTRATOR
4. EXIT

==========

Enter your choice (1-4):4

Program Exited!

Key Implementation Outlines Of The System

The Key Implementation Outlines of the Canteen Management System are as follows:

- The implementation of the Canteen Management System, where the Python programming language and MySQL database are integrated for the efficient management of customers, products, and sales transactions.

- The design of the database, where MySQL database management system is used for designing the database. Separate tables are created for customers, products, purchases, and product reports. The database consists of specific attributes for each table, where primary keys and foreign keys are used.

- The frontend of the system, where a command-line interface (CLI) is used for designing the frontend of the system. The Python programming language is used for designing the frontend. The system consists of a menu-driven structure, where the administrator can navigate through different modules of the system.

- The application logic, where Python programming language is used for designing the application logic of the system. Python programming language consists of specific functions that perform specific operations, where SQL queries are executed.

- The sales and billing module, where the customers and products are integrated for efficient management of sales and billing of the canteen.

Challenges faced

1. Preventing data leaks.
2. Limiting access depending on varied user roles.
3. Invoice generation disruptions.
4. Handling errors using the try-except methodology.
5. Preventing duplicate data entry.
6. Efficient indexing to maintain performance.
7. Making it efficient.
8. Taking care of intricate technicalities when combining the menus altogether.
9. Maintaining the consistency of data.

Learning and key takeaways

1. Learning to create tables for inventory, users, transactions and biling.
2. Understanding of important MySQL concepts like primary key, foreign keys and relationships.
3. Learning to connect Python to SQL which forms the core of our programme.
4. Improved and refined skills in writing clean, modular and reusable codes.
5. Integration skills become enhanced.

Future Enhancement

- **Enhanced Reporting:** Implement reports for **Low Stock Alerts** (displaying products below a set quantity threshold) and **Profit/Loss Analysis** (calculating revenue vs. cost).
- **Security Implementation:** Add a proper **User Authentication** and role-based access control system (Admin vs. Clerk) to directly address the system's **Security** NFR.
- **Performance Optimization:** Integrate support for **Barcode Scanning** into the sales module to drastically speed up transaction processing and improve input accuracy.

Conclusion

The **Canteen Management System** successfully fulfilled its objectives by delivering a robust, automated operational tool for the CSD, built on **Python** and a **MySQL** database. This project's strength lies in its adherence to the **Three-Tier Architecture**, which cleanly separates the Presentation (CLI), Logic, and Data layers. This design choice ensures the system is inherently **maintainable** and scalable for future enhancements.

The system now offers comprehensive **CRUD functionality** across the three major operational modules: Customer, Product, and Sales. A key achievement was implementing the complex, synchronized database logic within the **Sales Module**, which ensures that transactions are instantly logged while **concurrently adjusting the product inventory** in real-time. This eliminates the manual errors and stock discrepancies that were the core problems addressed by the system.

Ultimately, the Canteen Management System provides the administration with the necessary tools from accurate inventory tracking and automated transaction processing to detailed **invoice generation** to operate with greater efficiency and transparency, ensuring the continued effective service to Defence Personnel and their families. This serves as a strong platform demonstrating practical application of relational database principles and modular programming.

Bibliography

1. Python Programming: An Introduction to Computer Science" by John Zelle
2. MySQL Documentation: https://dev.mysql.com/doc/
3. scribd.com
4. Slideshare.com